```
$ kubectl apply -f complete-demo.yaml

$ kubectl -n sock-shop port-forward svc/front-end 8888:80
```

Creates all necessary Kubernetes resources (deployments, services, etc.) that together make up the app; if you want to keep an eye on it, waiting until all resources are ready, you can (in a different terminal session) use the watch kubectl -n sock-shop get deploy,pod,svc command.

Once all pods are ready, you can make the frontend accessible on your local machine using the port-forward command.

If you successfully set up the Sock Shop app, you can access the entry point (the `front-end` service) in your browser. Head over to http://localhost:8888, and you should see something akin to what is depicted in figure 1.3.

Next, you want to generate some traffic, so log in (in the right-hand upper corner) with the username `user` and the password `password`, and then add some items to your shopping cart. Now, what about observability? Let's have a look at a concrete example end to end.

The Sock Shop example app emits various signals. Let's take a closer look at two signal types, logs and metrics, to explore what exactly is emitted and what would be necessary to introduce observability for a given role.

In terms of the logs signal type, Sock Shop services emit a variety of logs. One example, looking at the `orders` service, is as follows (output edited to fit):

```
$ kubectl logs deploy/orders
2021-10-25 14:33:10.649  INFO [bootstrap,,,] 7 --- [           main] s.c.a.
AnnotationConfigApplicationContext : Refreshing org.springframework.context.
annotation.AnnotationConfigApplicationContext@51521cc1: startup date
[Mon Oct 25 14:33:10 GMT 2021]; root of context hierarchy
2021-10-25 14:33:16.838  INFO [bootstrap,,,] 7 --- [           main] tratio
Delegate$BeanPostProcessorChecker : Bean 'configurationPropertiesRebinderAu
toConfiguration' of type [class org.springframework.cloud.autoconfigure.Con
figurationPropertiesRebinderAutoConfiguration$$EnhancerBySpringCGLIB$$b894f
39] is not eligible for getting processed by all BeanPostProcessors (for ex
ample: not eligible for auto-proxying)
...
```

In this case, the source is the `orders` service, and the signal type is logs.

> **NOTE** Nearly everyone uses logs (in contrast to metrics or even traces), yet there is much less standardization and interoperability going on in this signal type compared to others. While OpenTelemetry is, at the time of publication, stable for traces and almost stable for metrics, we're still working in the community to achieve a similar level of coverage for logs.

To make the logs actionable, for a developer to fix a bug, for example, we would need the following:

- An agent (like Fluent Bit) to route the logs to a destination
- A destination, such as OpenSearch or CloudWatch